NIDAR AirMouse — Team Learning Roadmap & ESP32 Practice-Drone Guide

Team: 8 members (6 CS, 2 ENTC, 1 CAD-capable) · **Today:** Sep 26, 2026 · **Your documentation target:** Oct 23, 2026 · **Final Mission:** ~Jan 2027

This doc is written so anyone on the team can open it cold and know what to do next. Share this link with everyone.

---

1. What you're actually building (mission recap)

NIDAR AirMouse = an **autonomous indoor drone** that, with zero human piloting input:

1. Takes off from a 2ft×2ft launch zone and enters a maze (up to 15m×15m, corridors ≥1m wide, ≥8ft ceiling, 2m×2m rooms) through one entry point.
2. Navigates corridors/turns/junctions **without GPS** (GPS-denied, indoor).
3. Detects up to 6 survivors and tags their grid location.
4. Builds and continuously streams a live 2D map to a Ground Control Station (GCS).
5. Exits through a designated exit point — all within 30 minutes, with **zero manual intervention** (that includes no FPV goggles, no joystick correction, no manual tagging).

Key hard constraints:

- **No ready-to-fly/commercial airframes allowed** — you must design and build the frame/system yourselves.
- All-up weight ≤ 10 kg (this gives you room — you are not building a toy-sized drone for the real mission).
- Propeller guards are mandatory.
- No GSM/WiFi-internet/cloud during the mission — only your own local radio link.
- Must have failsafes: low battery, link loss, geofence breach, mission abort, emergency recall.
- Scoring (600 of 1000 pts) is dominated by: survivor detection+localization (240), 2D mapping accuracy (220), GCS map display (50), autonomous exit (50). Design Review (200 pts) separately scores your navigation strategy, mapping quality, and detection accuracy *on paper* — so your Oct submission needs to show a credible technical plan even before you've flown anything.

Bottom line: this is really three engineering problems stacked together — (a) a flying, stable multirotor, (b) autonomous GPS-denied navigation + real-time mapping, (c) onboard human/survivor detection. Nobody on a first-year team builds all three well by flying blind — hence the ESP32 practice drone below, which teaches (a) fast and cheaply while the rest of the team studies (b) and (c) in parallel.

---

2. Suggested team split (8 people)

Sub-team	People	Focus
Airframe & CAD	1 (CAD member) + 1 CS	3D-print frame design, weight/thrust balance, propeller guards, mounting for electronics
Flight & Electronics	2 CS + 1 ENTC	Motors, ESCs/drivers, battery, power system, flight controller firmware, PID tuning
Autonomy & Mapping	2 CS	SLAM/mapping, obstacle sensing (ultrasonic/ToF/LiDAR/camera), path planning
Vision (survivor detection)	1 CS	Camera + lightweight object/person detection model, grid-coordinate tagging
GCS / Comms / Docs	1 ENTC	Ground Control Station UI (map + camera feed + telemetry), radio link, and owns the Oct 23 documentation

Everyone should still go through Section 3 basics — you can't debug a system you don't understand at all, and the Design Review (Phase 4A) is graded on the *whole team's* understanding, not just one person's slides.

---

3. Learning path (do this in parallel with building)

3.1 Drone fundamentals (everyone, week 1)

- **NPTEL — Drone Systems and Control** (IISc Bangalore, Prof. Suresh Sundaram) — free, government-run, covers aerodynamics, 6-DoF modeling, sensors, Kalman filtering, path planning (RRT/A*), obstacle avoidance, and an intro to SLAM. This is the single best free course for your exact use case: https://onlinecourses.nptel.ac.in/noc25_ae30/preview
- **Joshua Bardwell (YouTube)** — the standard reference for how multirotors physically work, ESCs, motors, flight-controller firmware, PID tuning, building from parts. Watch his beginner build-series playlists even if you never use Betaflight: https://www.youtube.com/@JoshuaBardwell
- Skim: how a quadcopter achieves roll/pitch/yaw/throttle from 4 motors (X-frame mixing) — this concept carries over to your ESP32 drone and the real one.

3.2 Flight controller firmware / autopilots (Flight & Electronics sub-team)

- **ArduPilot docs** (open-source autopilot, what most non-commercial teams use): https://ardupilot.org/copter/
- **PX4 docs** (alternative open-source autopilot, ROS-friendly): https://docs.px4.io/main/en/
- Note: for your *real* AirMouse drone you'll very likely run ArduPilot or PX4 on a proper flight-controller board (or your own board) rather than the toy ESP32 firmware — the ESP32 drone is a stepping stone to understand concepts, not the final flight stack.

3.3 GPS-denied navigation & indoor mapping (Autonomy sub-team — this is your hardest problem)

- Search terms to study: **visual-inertial odometry (VIO)**, **optical flow sensors**, **2D LiDAR SLAM**, **ORB-SLAM3**, **RTAB-Map**, **Cartographer**.
- Practical low-cost sensor options used by real indoor drones: a downward optical-flow + ToF sensor pair (like the PMW3901 + VL53L1X combo used in Crazyflie/ESP-Drone extension boards) for height/position hold, plus a forward-facing depth camera or 2D LiDAR (e.g. RPLiDAR A1) for mapping and obstacle avoidance.
- **ArduPilot's non-GPS navigation guide** is directly relevant: https://ardupilot.org/copter/docs/common-non-gps-navigation-landingpage.html
- **JdeRobot RoboticsAcademy** — a free, ready-to-use online course specifically on autonomous drone programming (follow-line, visual landing, search-and-rescue exercises) run in simulation, no hardware needed to start learning: https://jderobot.github.io/RoboticsAcademy/
- Read the ArduPilot forum thread "vSLAM with ArduPilot" for a real team's trial-and-error notes on exactly your problem (indoor visual SLAM + ArduPilot): https://discuss.ardupilot.org/t/vslam-with-ardupilot/105395

3.4 Survivor detection (Vision sub-team)

- Use a lightweight person-detection model (YOLOv8-nano / YOLO-tiny or MobileNet-SSD) running on a companion computer (Raspberry Pi 4/5 or Jetson Nano/Orin Nano) — full-size YOLO won't run fast enough onboard.
- Free resources: Ultralytics YOLOv8 documentation and quick-start (https://docs.ultralytics.com/), and OpenCV's free "Getting Started with OpenCV" course material for the camera pipeline.
- You don't need to detect "survivors" specifically — a person-detector plus mapping the detection back to the drone's estimated pose (and hence the arena grid cell) is exactly what's scored.

3.5 Ground Control Station

- You need a live map + camera feed + telemetry display. **QGroundControl** (PX4/ArduPilot compatible, open source) is a fast way to get a working GCS baseline, or your GCS sub-team can build a custom one (e.g. with a simple Python/Qt or web dashboard) if you need custom map/survivor-tag overlays: http://qgroundcontrol.com/

---

4. Build the $25–40 ESP32 practice drone first

Goal: get something in the air that can go up/down/hover/drift sideways within 1–2 weeks, so the whole team internalizes motors→ESC/driver→flight-controller→IMU→PID loop before touching the real, much bigger AirMouse airframe. This is a *learning rig*, not your competition drone.

4.1 Two proven open-source paths (pick one)

Option	What it is	Why pick it
A. Espressif ESP-Drone	Espressif's own official open-source nano-drone firmware for ESP32, ported from the Crazyflie project. Controlled via a phone app over WiFi.	Most "official"/well-documented; has extension boards for height-hold/position-hold later. Repo: https://github.com/espressif/esp-drone · Docs: https://docs.espressif.com/projects/espressif-esp-drone/en/latest/
B. cifertech ESP32-Drone	Community Arduino-based firmware: MPU6050 IMU + Madgwick filter + PID stabilization, controlled from a **browser** over WiFi (no app install needed) — easiest for a CS-heavy team to read and modify since it's plain Arduino C++. Repo: https://github.com/cifertech/ESP32-Drone	Simpler codebase to actually understand and hack on; good if your goal is "learn how PID + IMU fusion works," not just fly.

Also worth skimming for BOM/wiring reference (very similar builds, well documented, ₹1,000–1,500 total cost reported):

- CircuitDigest "DIY WiFi-Controlled Drone with ESP32 and Arduino": https://circuitdigest.com/microcontroller-projects/DIY-wifi-controlled-drone
- Hackster.io "How to Make a Cheap ESP32 Drone Under $15": https://www.hackster.io/e_s_c/how-to-make-a-cheap-esp32-drone-under-15-371767

4.2 Bill of materials (approximate, budget build)

Part	Qty	Notes	Approx. cost (₹)
ESP32-WROOM-32 dev board	1	Any generic dev board works	250–350
MPU6050 IMU module	1	Gyro+accel, I2C	80–120
716/720-size coreless DC motors	4	CW×2, CCW×2 (or just flip 2 props)	60–100 each → 300–400
55mm 2-blade plastic props (matched CW/CCW pairs)	2 sets	Must match motor rotation direction	80–150
N-MOSFETs (e.g. SI2302/AO3400) or a small 4-in-1 brushed motor driver	4 or 1 board	To PWM-switch each brushed motor from ESP32 GPIO (do	40–100

Part	Qty	Notes	Approx. cost (₹)
		not drive motors directly off GPIO pins)	
1S LiPo battery, 300–600 mAh	1	Match to frame weight; smaller pack = lighter but shorter flight	150–250
TP4056 Li-ion/LiPo charging module	1	For safe charging	30–50
Schottky diode + basic passives	few	Power protection	20
Thin hookup wire, header pins, double-sided foam tape	—	Assembly	50
3D-printed frame	1	See 4.3 below — your own design, print it yourselves	~free (dept. printer)
Total			**≈ ₹1,000–1,500**

Skip ESCs entirely — coreless brushed motors are switched directly by a MOSFET + PWM, which is exactly why this build is cheap. (Real drones for the competition use brushless motors + ESCs — different from this practice rig.)

4.3 3D-printed frame in PLA — what actually matters

PLA is brittle, not strong under vibration/impact — but for a **sub-60g micro-drone with tiny coreless motors**, the actual forces involved are small, so PLA is genuinely fine as long as you design around its weaknesses:

- **Keep it light and simple:** an "X" or "H" frame with thin arms (aim for total printed frame weight under 8–10g). Use infill of only 15–20% — you want light, not solid.
- **Print arms along the length, not across:** PLA is weakest at layer lines: orient the print so arms are printed lengthwise (layers running along the arm) to resist the bending force from motor thrust, not across it (which snaps easily).
- **Motor mounts:** print a small boss/collar sized exactly to your motor's diameter, with a slight interference fit or a couple of tiny screws — the motors are so light that press-fit + a drop of glue is usually enough.
- **Vibration isolation:** even simple: a thin strip of foam tape between the ESP32/IMU board and the frame reduces vibration noise into the MPU6050, which otherwise makes the drone twitchy.
- **Battery mount:** velcro or a foam-tape "pocket" on top or bottom, positioned so the center of gravity sits at the geometric center of the 4 motors — CoG offset is the #1 reason first-time micro-builds won't hover stable.

- Expect 2–3 print iterations. Print your first frame *skeleton-only* (arms + center plate, no covers) to test motor spacing and CoG before committing to a "final" design.

4.4 Assembly & first-flight order of operations

1. Print frame → dry-fit motors and board without wiring, check CoG by balancing on a finger under the board's center.
2. Wire ESP32 GPIOs → MOSFETs → motors (never GPIO → motor directly; the ESP32 pins can't source the current).
3. Wire MPU6050 to ESP32 via I2C (SDA/SCL), flash the chosen firmware (ESP-Drone or cifertech ESP32-Drone) using Arduino IDE / ESP-IDF.
4. Bench test **with propellers removed first**: verify each motor spins the right direction from the app/browser control before ever attaching props.
5. Calibrate IMU on a flat, level surface (both firmwares have a calibration step — don't skip it).
6. First flights: **indoors, low ceiling, over a soft mat, one team member controlling, everyone else phones away** — micro coreless drones fly erratically for the first several attempts while you tune PID gains. That's expected and is exactly the learning point.
7. Once stable hover + basic directional control works, you've validated the full loop (sensor → filter → PID → motor mixing) that scales up conceptually to the real AirMouse drone.

4.5 Safety notes (still apply even to a toy-sized drone)

- Always fly with props removed for any first-time code/wiring test.
- LiPo/Li-ion cells: never charge unattended, use the TP4056 as intended, don't puncture or short cells.
- Keep fingers clear of spinning props even at this small scale — coreless motors are low-torque but props still nick skin.

---

5. From practice drone → actual AirMouse competition drone

The ESP32 rig teaches concepts; the competition drone is a different, larger build. Rough gap list for your Design Review documentation:

Practice drone	Competition drone needs
Coreless brushed motors, no ESCs	Brushless motors + ESCs, sized for a larger frame + payload (companion computer, sensors, battery) within the 10kg limit
ESP32 running toy flight firmware	A real flight controller running ArduPilot or PX4 (or a custom-built one, since ready-made *airframes* aren't allowed — check whether a standard flight-controller board is acceptable, and clarify custom-hardware requirements against Rulebook clause 8.2)
Phone/browser WiFi control	A dedicated local radio link (no internet/cloud) between drone and your custom GCS, per the "no external network" rule

Practice drone	Competition drone needs
No mapping/vision	Companion computer (Raspberry Pi/Jetson) running SLAM + a lightweight person-detector, feeding a live map + camera feed to your GCS
No propeller guards	Mandatory full propeller guards
No failsafes	Low-battery, link-loss, geofence, and mission-abort failsafes wired into the flight stack

Your Oct 23 documentation should present this as a deliberate staged plan (Section 6) rather than apologize for not having the full system yet — the rulebook's own Progress Review Rounds (Oct, Dec) exist precisely because no team has this fully working in September.

---

6. Suggested timeline

Window	Milestone
Sep 26 – Oct 5	Everyone completes Section 3.1 basics; sub-teams start their specific learning tracks (3.2–3.5); CAD member starts frame v1 design
Oct 5 – Oct 15	ESP32 practice drone built, bench-tested, first hover attempts; Autonomy/Vision sub-teams have a working SLAM or detection demo *in simulation or on a laptop*, even without a flying platform yet
Oct 15 – Oct 22	Draft the documentation: mission architecture diagram, sensor/hardware choices with justification, navigation + mapping approach, survivor-detection approach, safety/failsafe plan, rough cost sheet
Oct 23	Submit documentation
Oct (2nd week) →	Progress Review Round 1 — expect to present development status; keep iterating on the real airframe design from here through the Dec review and into the Jan final

---

7. Quick link list

- NPTEL Drone Systems and Control (free): https://onlinecourses.nptel.ac.in/noc25_ae30/preview
- Joshua Bardwell (YouTube): https://www.youtube.com/@JoshuaBardwell
- ArduPilot docs: https://ardupilot.org/copter/
- ArduPilot non-GPS navigation: https://ardupilot.org/copter/docs/common-non-gps-navigation-landingpage.html
- PX4 docs: https://docs.px4.io/main/en/

- JdeRobot RoboticsAcademy (free drone-programming course): https://jderobot.github.io/RoboticsAcademy/
- Espressif ESP-Drone: https://github.com/espressif/esp-drone
- cifertech ESP32-Drone (Arduino, browser control): https://github.com/cifertech/ESP32-Drone
- CircuitDigest ESP32 drone build guide: https://circuitdigest.com/microcontroller-projects/DIY-wifi-controlled-drone
- Hackster $15 ESP32 drone: https://www.hackster.io/e_s_c/how-to-make-a-cheap-esp32-drone-under-15-371767
- Ultralytics YOLOv8 docs (for survivor/person detection): https://docs.ultralytics.com/
- QGroundControl (GCS): http://qgroundcontrol.com/